

Exploring SARSA vs. Q-Learning on the Indiana Jones' Adventure Puzzle

Cesar Gonzales
EN.705.741 Reinforcement Learning
Johns Hopkins University
March 24, 2026

I. INTRODUCTION

A. Hypotheses

Temporal difference (TD) learning refers to a class of Reinforcement Learning (RL) methods that are well suited to online learning situations [1]. Unlike Monte-Carlo RL methods, which delay any learning (i.e., value function updates) until the end of episodes, 1-step TD learning methods need only wait a single time step before the next update can occur. SARSA and Q-Learning are two prominent RL algorithms that fall under the TD family of methods. The former uses on-policy value function updates, while the latter uses off-policy updates. We will explore how these algorithms compare when applied on a 2D grid world puzzle where a RL agent must learn to find the optimal path from some starting location to a goal location. This type of problem is similar to the “Cliff Walking” problem described by Sutton and Barto in [1], where an agent must learn to find the optimal path from a starting point to a goal point while avoiding some designated cliff space that leads to highly negative rewards. Ultimately, Sutton and Barto demonstrated that a SARSA RL agent yields higher returns per episode compared to a Q-Learning agent on this task [1]. That said, the Q-Learning agent learns the optimal (yet more dangerous) path to the goal point while the SARSA agent learns a more roundabout (yet safer) path. The authors propose that gradually reducing the value of epsilon used for e-greedy action selection will allow both types of agents to learn the optimal path asymptotically [1]. Considering this, comparable results should be expected when comparing SARSA vs. Q-Learning agents that learn to play a similar 2-D grid world puzzle. Specifically, we believe that a SARSA RL agent will outperform a Q-Learning RL agent on the “Indiana Jones' Adventure” 2-D puzzle in terms of return per episode during learning. However, we predict the Q-Learning agent will learn the optimal policy, while the SARSA agent will learn a more conservative but safer policy.

II. METHODS

A. The “Indiana Jones' Adventure” Puzzle

The problem is dubbed the “Indiana Jones' Adventure” puzzle in reference to the iconic American media franchise about a fictional archaeologist adventurer who is named Dr. Henry Walton “Indiana” Jones, Jr. [2]. The puzzle includes a background story as follows: The hero, Indiana Jones, must race against time to obtain a valuable antiquity and deliver it to a museum. The puzzle begins with Indiana Jones starting at a location denoted by ‘ α ’ on his map after he has managed to enter a locked temple where the antiquity resides. Once Indiana Jones retrieves the antiquity, he will immediately fall through a trap door and be transported outside the temple denoted by a location marked ‘ β ’ on his map. After this, Indiana Jones can either walk through an unstable plank and rope bridge over a swamp or take an alternate more roundabout route to get to the museum. If traversing the former, there is a risk of falling into the swamp, which results in having to swim back out of the swamp to re-emerge at the ‘ β ’ location. Thus, this route is potentially much slower and more costly since time is critical. The puzzle is solved once Indiana Jones reaches the museum. The associated game board and token descriptions for the puzzle are shown in Fig. 1 and 2.

B. The Corresponding RL Problem and Environment Dynamics

In a RL context, Indiana Jones corresponds to a learning agent and the game board depicted in Fig. 1 is his surrounding environment. As you can see, the game board is a 6x16 grid where each grid cell is a state defined by a corresponding integer label i where $0 \leq i \leq 95$. This integer label specifies the state's location within the grid. Thus, there is a maximum of 96 possible states. The agent begins in state 0 (i.e., the ‘ α ’ location within the temple), and the museum is at state 95. Note that the museum state is the only terminal state in this scenario. The state transition function is deterministic and includes the following caveats: the transition out of the temple (after retrieving the antiquity) into location ‘ β ’ occurs instantaneously. In other words, the environment will never report that the agent is at the antiquity state but will instead report that that agent is at state ‘ β ’. The same goes for any of the swamp states. At each time step, the agent can take one of four possible actions that are encoded as integers: move North (0), move South (1), move West (2), or move East (3). Any action that would cause the agent to collide with the game board's boundaries results in no change in the agent's current state (i.e., the same as if no action was performed). Transitions into the antiquity state results in a reward of 4. Transitioning into any of the swamp states yields a reward of -100. Any other transitions yield a reward of -1.

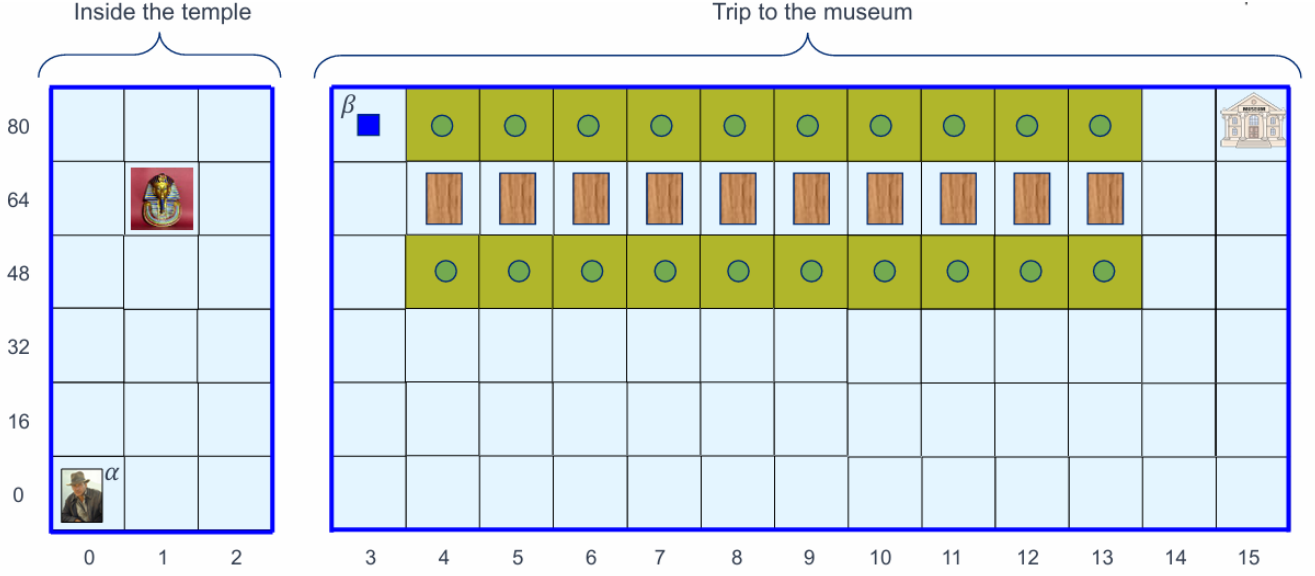


Fig. 1: The Indiana Jones' Adventure game board. Indiana Jones starts at state 0 and must retrieve the antiquity at state 65. He is then immediately transported to the β state (i.e., state 83), after which he must get to the museum (i.e., state 95) while avoiding the swamp states.

	Indiana Jones, the archeologist.
	The rare antiquity that Indy is seeking
	The museum (terminal state)
	Empty (traversable) space
	Swamp space
	Rope and plank bridge space
	Location α
	Location β

Fig. 2: Descriptions of the tokens shown in the game board in Fig. 1.

C. Agents

Two types of agents were implemented to try to solve this puzzle: a SARSA RL agent and a Q-Learning RL agent. Both types of agents relied on ϵ -greedy action selection with epsilon initially set to $\epsilon = 0.1$. Note that these two types of learning agents are very similar, and the primary difference between them is that SARSA is an on-policy algorithm while Q-Learning is an off-policy algorithm. More specifically, the two algorithms differ in how the updates of the action value function $\hat{q}(s, a)$ are performed. The SARSA algorithm updates its $\hat{q}$ values by using an action for a future state such that the action was chosen using ϵ -greedy action selection. Thus, the same policy is used for both action selection and learning updates. In contrast, Q-Learning always uses the action in some future state that leads to the maximum $\hat{q}$ function value in that future state. Thus, the Q-Learning algorithm uses one policy (i.e., a behavior policy) for action selection but another policy (i.e., a target policy)

for learning updates. The $\hat{q}$ function update equations for each agent type at time step t are as follows:

SARSA:

$$\hat{q}(S_t, A_t) = \hat{q}(S_t, A_t) + \eta [R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}) - \hat{q}(S_t, A_t)]$$

Q-Learning:

$$\hat{q}(S_t, A_t) = \hat{q}(S_t, A_t) + \eta [R_{t+1} + \gamma \max_a \hat{q}(S_{t+1}, a) - \hat{q}(S_t, A_t)]$$

where η is the learning rate, R_{t+1} are the immediate rewards, and γ is the discount factor.

D. Agent Training

In total, 20 instances for each type of agent were trained, each of them for 500 episodes. Each agent instance used undiscounted future rewards during the $\hat{q}$ function updates. I.e., the discount factor was set to $\gamma = 1$. The learning rate was set to $\eta = 0.4$ and remained constant throughout training. The behavior policy was ϵ -greedy action selection for all agents. Two different experiments were performed with regards to ϵ -greedy action selection. The first experiment used a fixed epsilon value $\epsilon = 0.1$ for all training episodes. The second experiment linearly decayed the epsilon value from $\epsilon = 0.1$ to $\epsilon = 0.02$ during the first 200 training episodes. Decaying the epsilon value allows the agents to increasingly exploit what they have already learned as training progresses.

E. Data Collection and Metrics

Each agent's performance was measured using the total accumulated rewards (i.e., the return) per episode. For each agent type, every episode's average return was computed across all the agents of that type. That is, the following sum was computed for both types of agents:

$$G_i = \frac{1}{N} \sum_{j=1}^N G_{i,j} \quad 1 \leq i \leq 500$$

where $N = 20$ is the number of agent instances for each type. Averaging over the agents mitigates the variability induced by differences between the agents' decision policies as they learn to play the Indiana Jones' Adventure puzzle. Learning curve plots were collected to evaluate each type of agent's learning experiences. To minimize data noise, a simple smoothing (i.e., averaging) function was applied, which is defined by the following equation:

$$\bar{G}_i = \frac{1}{w} \sum_{k=i-\lfloor w/2 \rfloor}^{i+\lfloor w/2 \rfloor} G_k \quad 1 \leq i \leq 500$$

where w is the window size. Note that this smoothing function is not clearly defined when applied to the boundaries of the data points collected. That is, the smoothing window w will extend beyond the collected data points whenever $i < w/2$ and $i > 500 - w/2$. Even if the data were to be padded at the boundaries, any smoothed points within these ranges are not valid. To address this, the smoothing of these points is simply excluded from the plots. Thus, each of the learning curves plots only depicts the smoothed data between $w/2 < i < 500 - w/2$. For visual purposes, the smoothed data is shifted to the left by $w/2$ points so that the first smoothed point starts at zero along the x-axis. To ensure at least 500 data points are represented, the last $w/2$ data points are repeated twice at the tail end of the smoothed data. However, only the first $500 - w$ points are actually valid. A window size of $w = 11$ was used for all collected learning curve plots, meaning the repeated data is minimal. Following the computation of the smoothed average return G_i across the 20 agents for each learning agent type, the total average return per episode was computed as follows:

$$\bar{G} = \frac{1}{500} \sum_{i=1}^{500} G_i$$

This allowed for a direct quantitative comparison of learning performance between the two agent types. Note that all the data collection methods described above were performed twice, with and without decay-epsilon-greedy action selection.

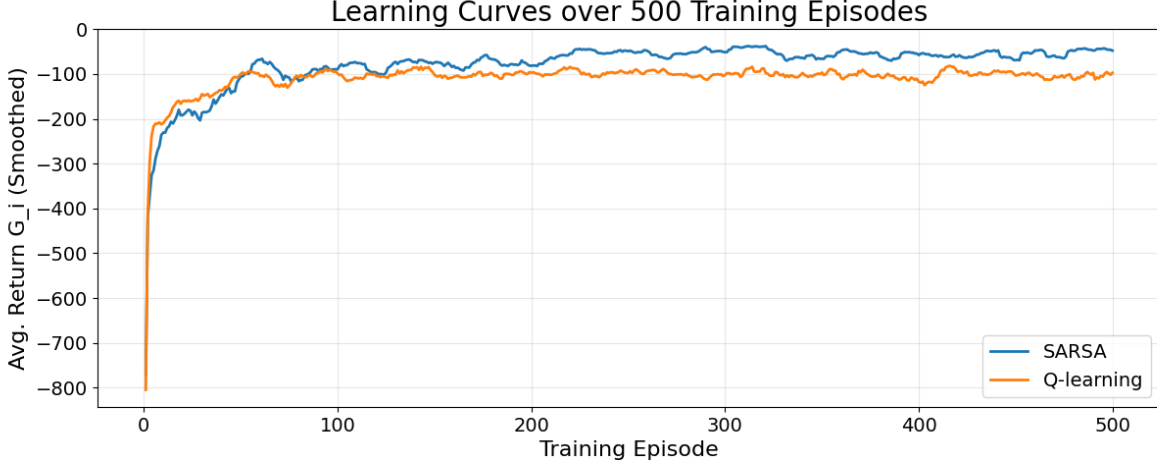


Fig. 3: The learning curves of the SARSA and Q-Learning RL agents over 500 episodes. For each agent type, the return per episode was averaged across all 20 agent instances. Additionally, smoothing was applied to minimize the data noise.

Agent Type	G_{avg}	Std Deviation
SARSA	-90.0	194.72
Q-Learning	-121.5	213.91
SARSA (epsilon decay)	-55.3	143.61
Q-Learning (epsilon decay)	-45.2	163.64

TABLE I: The average return for each agent type over 500 training episodes. The last two rows correspond to the results of using decayed-epsilon-greedy action selection.

III. RESULTS

A. Experiment 1 (Fixed ϵ value)

As mentioned in the previous section, the epsilon value used for ϵ -greedy action selection remained constant throughout this experiment. From the learning curves in Fig. 3, it is observed that the SARSA agents outperform the Q-Learning agents on average during training. The total average returns in table I confirm this. The policies learned by one instance of each type of agent are shown in Fig. 4 and 5, respectively. Clearly, the Q-Learning agent learns a better policy that will lead Indiana Jones to the museum more quickly than the SARSA policy. Thus, despite under-performing during the training episodes, the Q-Learning agent ends up learning the optimal policy while the SARSA agent learns a more conservative but costly policy. One might ask why then the Q-Learning agents performed worse than the SARSA agents during training. The reason is due to differences in the $\hat{q}$ function updates and agent exploration during learning. That is, the Q-Learning agents tend to favor greedy state transitions more than the SARSA agents because of the way the next state's actions are used for updating the $\hat{q}$ function. I.e., in Q-Learning, the action of the next state used for learning updates is always the one that leads to the maximum $\hat{q}$ value

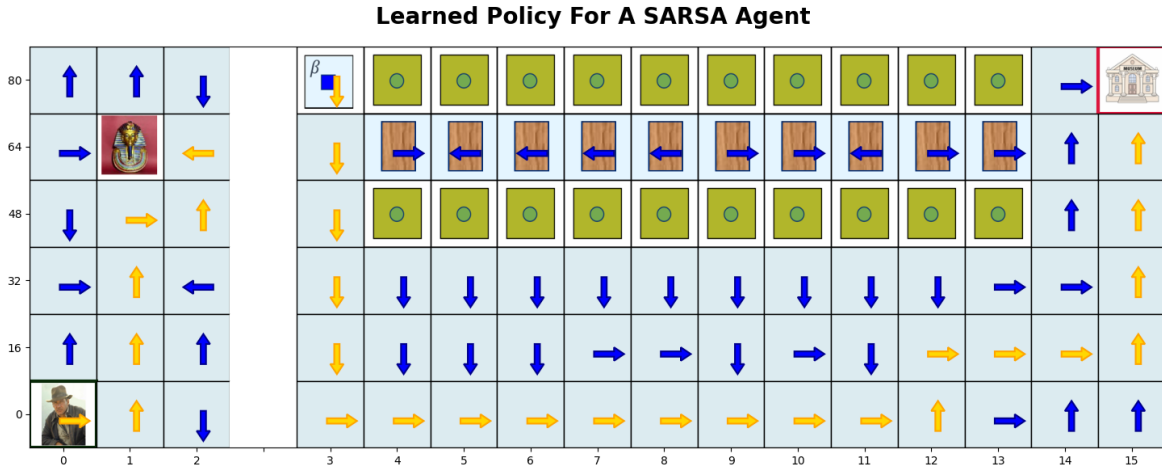


Fig. 4: The learned policy for one of the trained SARSA agents on the Indiana Jones' Adventure puzzle.

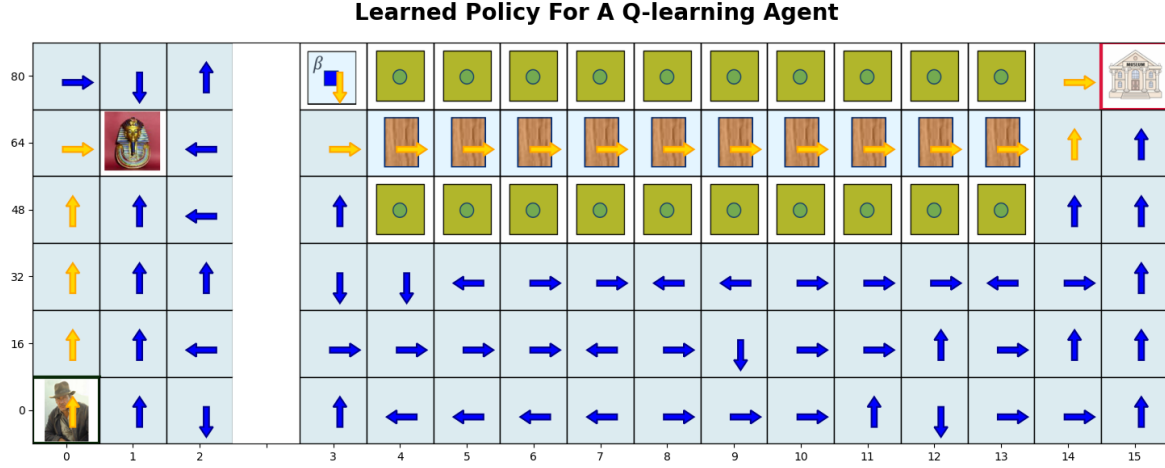


Fig. 5: The learned policy for one of the trained Q-Learning agents on the Indiana Jones' Adventure puzzle.

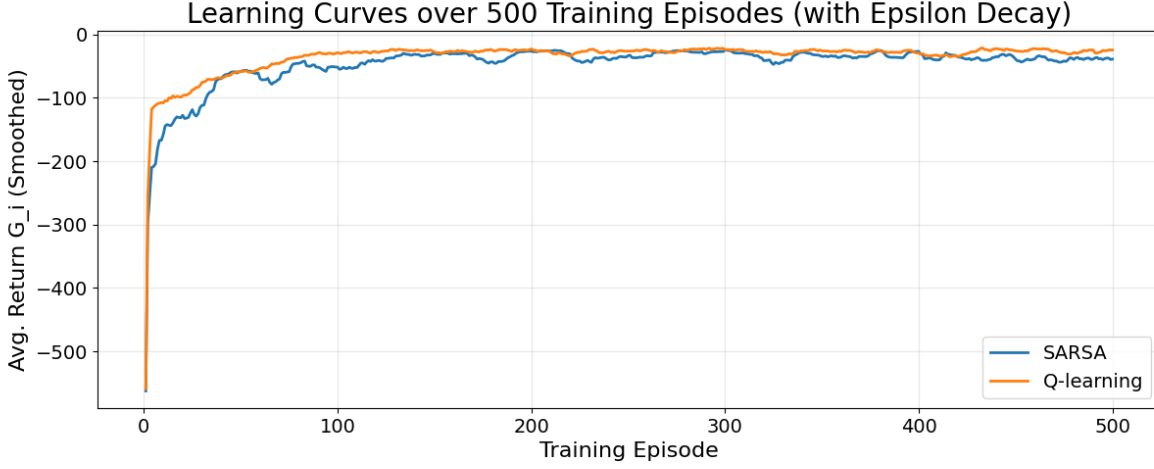


Fig. 6: The learning curves of the SARSA and Q-Learning RL agents over 500 episodes. Here, the epsilon value used for ϵ -greedy action selection was linearly decayed during the first 200 episodes. For each agent type, the return per episode was averaged across all 20 agent instances. Additionally, smoothing was applied to minimize the data noise.

for that state. Consequently, the Q-Learning agents end up spending more time around the swamp states than the SARSA agents because their $\hat{q}$ function updates led them there. Since ϵ -greedy action selection is performed, there is some non-zero probability of choosing to fall into the swamp states. Thus, the inherent greediness of Q-Learning's target policy is what leads the Q-Learning agents to perform worse on average. Meanwhile, the SARSA agents learn to play more conservatively and end up actively avoiding the swamp states that lead to highly negative rewards. This explains why the SARSA policy in Fig. 4 exhibits a roundabout and longer route to the museum. These results are comparable with Sutton and Barto's findings in their experiments on the "Cliff Walking" problem [1]. One other point worth noting is that both policies in Fig. 4 and 5 include "infinite loop" transitions where the agent can get stuck between one or two states if either of these states are ever reached (e.g., state 80 in the SARSA policy and states 40 and 41 in the Q-Learning policy). This is possibly due to insufficient exploration and certain states that lack sufficient $\hat{q}$ value updates, as these states were rarely or never visited during learning.

B. Experiment 2 (Linearly-decreasing ϵ)

For this experiment, the epsilon value used for ϵ -greedy action selection was linearly decayed during the first 200 training episodes. Here, the Q-Learning agents outperform the SARSA agents on average during learning, as exhibited by the learning curves in Fig. 6 and as well as table I. Even though the Q-Learning agents still learn to spend more time around the swamp states, there is less probability of them choosing to fall into the swamp states due to the decreasing epsilon value. The policies learned by one instance of each type of agent in this experiment are shown in Fig. 7 and 8, respectively. Once again, the Q-Learning agent learns the optimal policy while the SARSA agent learns a conservative but more costly policy. Note, however, that the SARSA agent's policy for this experiment is even more roundabout and slightly more optimal than the SARSA policy

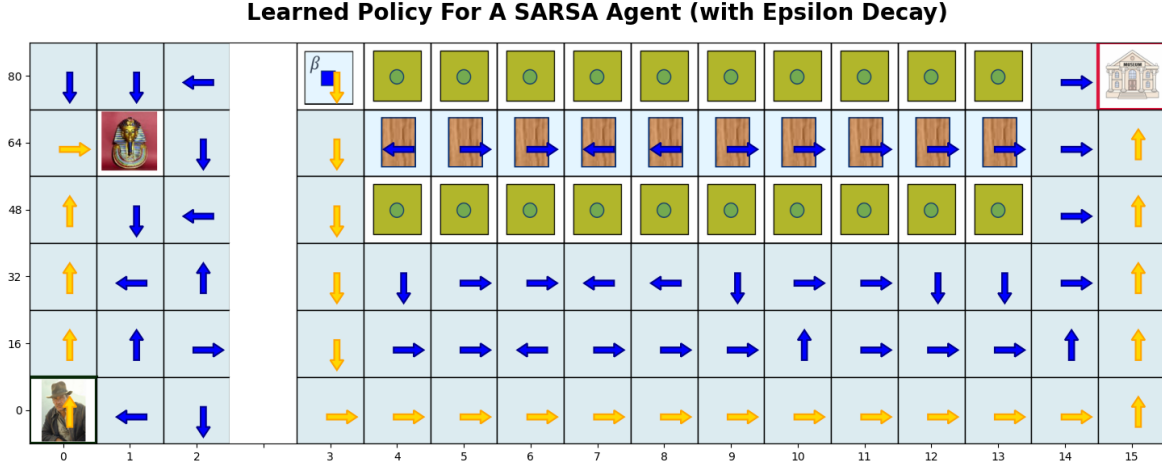


Fig. 7: The learned policy with decayed-epsilon-greedy action selection for one of the trained SARSA agents on the Indiana Jones' Adventure puzzle.

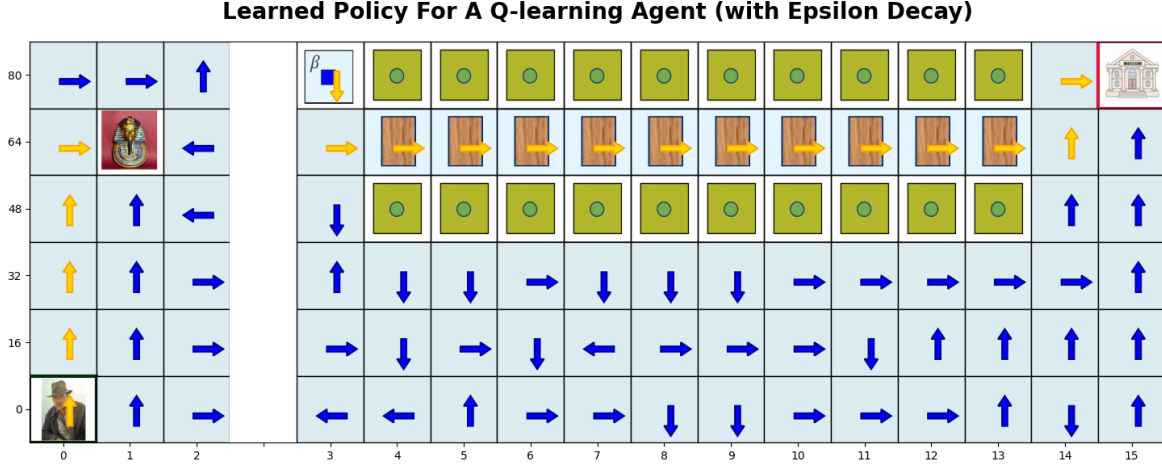


Fig. 8: The learned policy with decayed-epsilon-greedy action selection for one of the trained Q-Learning agents on the Indiana Jones' Adventure puzzle.

learned in experiment 1, particularly when Indiana Jones is located within the temple. Conversely, the Q-Learning agent policy in this experiment includes more “infinite loop” state transitions compared to the Q-Learning policy in experiment 1. This is likely due to decaying the epsilon value, which resulted in less exploration on average for the Q-Learning agents during this experiment. Regarding the average return per episode during training, both agent types in experiment 2 outperformed their counterparts in experiment 1. Using decay-epsilon-greedy action selection results in less exploration and more exploitation of learned “good” state transitions for both agent types. In other words, the RL agents in experiment 2 focused more on state transitions that led to higher cumulative rewards compared to the agents in experiment 1. Thus, as shown in table I, the experiment 2 agents yielded higher cumulative rewards on average compared to the agents in experiment 1.

IV. DISCUSSION

The results presented in the previous section have a few implications. One worth discussing is that SARSA agents tend to learn more conservatively compared to Q-Learning agents. This can be crucial in online learning scenarios where poor performance due to greedy $\hat{q}$ function updates is unacceptable. For example, consider a mobile robot attempting to learn an optimal route within its environment. It may be undesirable to allow the robot to keep traversing near potentially dangerous states (e.g., the swamp) for many episodes while it learns the optimal route. On the other hand, perhaps the greediness of Q-Learning is acceptable in certain situations (e.g., one in which time is the most critical factor). In this case, Q-Learning would be preferable over SARSA since the optimal policy could be found more quickly. As mentioned in the Introduction, both algorithms will asymptotically converge to the optimal policy over time given the value of epsilon used for e-greedy action selection is gradually reduced. This could be worth validating in a future experiment where the SARSA agents are trained on

the Indiana Jones' Adventure puzzle for a much larger number of episodes (e.g., 500,000 episodes). One could then determine whether SARSA does in fact converge to the optimal policy for this puzzle.

V. CONCLUSION

SARSA and Q-Learning are two effective temporal difference learning methods that can be applied to a variety of RL problems. Both methods perform particularly well on the Indiana Jones' Adventure puzzle. However, SARSA outperforms Q-Learning when evaluated on average returns per episode and using a constant epsilon value for ϵ -greedy action selection. If the epsilon value is linearly decayed throughout learning, then Q-Learning becomes superior. With regard to learning the optimal policy for this puzzle, Q-Learning clearly outperforms SARSA for either a fixed or decaying epsilon value. Even so, the conservative learning nature of SARSA may be more desirable in certain online learning situations.

REFERENCES

- [1] Sutton, R., Barto, A, "Reinforcement Learning: An Introduction", 2nd ed., pp. 123-132, The MIT Press, 2018
- [2] Indiana Jones and the Raiders of the Lost Ark. Directed by Steven Spielberg, Paramount Pictures, 1981.